

Packages & Tools Used in the Conway Order-7 Non-Existence Artifact

Official Sources and Reading Guide

This document lists every external package, binary, and format the artifact depends on, with the *official* website/repository for each and a pointer to the key paper or documentation to read to understand how it works. The mathematical claim is: every one of the 98,536 cubes is UNSAT (each backed by a machine-checkable LRAT certificate) and the cubes exactly tile the search space, hence no $\text{srg}(99, 14, 1, 2)$ admits an order-7 automorphism. The SAT solver is *untrusted*; trust rests only on the proof checkers and the coverage arithmetic — so the checkers below are the most important tools to understand.

At a glance

Tool / file	Role in this artifact	Official source
CaDiCaL (tools/cadical)	SAT solver; refutes each cube, emits LRAT	https://github.com/arminbiere/cadical
PySAT (python-sat)	CNF encoder library (conway_o7.py)	https://pysathq.github.io
lrat-check (tools/lrat-check)	LRAT checker (Heule) — trusted base	https://github.com/marijnheule/drat-trim
lrat-trim (tools/lrat-trim)	LRAT checker/trimmer (Biere)	https://github.com/arminbiere/lrat-trim
cake_lpr (planned)	Formally-verified LRAT checker	https://github.com/tanyongkiam/cake_lpr
DRAT/LRAT format	The proof format itself	https://github.com/marijnheule/drat-trim
Zstandard (zstd)	Compresses the 98,536 proofs & the kit	https://facebook.github.io/zstd/
Slurm	HPC job scheduler (the cluster run)	https://slurm.schedmd.com/
Apptainer / Singularity	Container for the cluster / re-run	https://apptainer.org/
Python 3	Runs encoder, coverage, aggregation	https://www.python.org/

1 SAT solving and proof certification (the core)

1.1 CaDiCaL — the SAT solver

The complete CDCL SAT solver by Armin Biere. In this artifact it refutes each cube and, run with `--factor=false`, emits a plain-RUP LRAT proof (`solve_cube.py --proof`). It is also invoked *inside* the encoder through PySAT’s `Cadical195` interface. Remember: CaDiCaL is **untrusted** here — if it lies, the checkers below catch it.

Official repository <https://github.com/arminbiere/cadical>

Project page (Biere / FMV) <http://fmv.jku.at/cadical/>

Read to understand it The repository README.md, and the tool paper: A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, F. Pollitt, “CaDiCaL 2.0” (CAV 2024), https://link.springer.com/chapter/10.1007/978-3-031-65627-9_7

1.2 PySAT (python-sat) — the CNF encoder library

The Python toolkit used by `conway_o7.py/export_o7.py` to build the DIMACS instance `o7.cnf`. It provides the cardinality encodings for the degree and common-neighbour constraints (the project selects the *modulo totalizer*, *mtotalizer*, via `CONWAY_DEG_ENC=mtotalizer`) and a uniform interface to CaDiCaL.

Documentation <https://pysathq.github.io>

Repository <https://github.com/pysathq/pysat>

PyPI package <https://pypi.org/project/python-sat/>

Cardinality encodings API <https://pysathq.github.io/docs/html/api/card.html>

Read to understand it A. Ignatiev, A. Morgado, J. Marques-Silva, “PySAT: A Python Toolkit for Prototyping with SAT Oracles” (SAT 2018) — linked from the docs homepage. For the encodings themselves see the *totalizer* / *modulo-totalizer* references on the `pysat.card` page.

1.3 The DRAT / LRAT proof format

LRAT (“Linear RAT”) is the machine-checkable UNSAT-proof format the whole certificate is built on: each line is a resolution/RAT step with explicit clause-id hints, so a checker verifies it in one linear pass without search. This is the format `cadical` emits and every checker below consumes. Understand this format *first* — it is the object of trust.

DRAT-trim / lrat-check home <https://github.com/marijnheule/drat-trim>

LRAT format paper L. Cruz-Filipe, M. Heule, W. Hunt, M. Kaufmann, P. Schneider-Kamp, “Efficient Certified RAT Verification” (CADE 2017).

DRAT background N. Wetzler, M. Heule, W. Hunt, “DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs” (SAT 2014).

1.4 lrat-check (Heule) — checker #1

The reference LRAT checker (`lrat-check.c`) that ships in the DRAT-trim repository. It is what `verify_final.sh` / `verify_proofs.sh` call to re-verify every certificate. Small, public, and replaceable — part of the trusted base.

Source (in DRAT-trim repo) <https://github.com/marijnheule/drat-trim/blob/master/lrat-check.c>

Read to understand it The header comments in `lrat-check.c`, plus the LRAT format paper above.

1.5 `lrat-trim` (Biere) — checker #2 / trimmer

Armin Biere’s independent LRAT reader/checker/trimmer. Using it *as well as* `lrat-check` means two independently-written checkers agree on every proof (defence against a bug in either one).

Repository <https://github.com/arminbiere/lrat-trim>

Read to understand it The repository README and the top of `lrat-trim.c`; background paper: F. Pollitt, M. Fleury, A. Biere, “Faster LRAT Checking Than Solving with CaDiCaL” (SAT 2023), <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SAT.2023.21>

1.6 `cake_lpr` (CakeML) — checker #3, formally verified (planned)

The strongest check on the to-do list: a checker whose x86-64 machine code is *itself* proved correct in the CakeML/HOL4 ecosystem. Because all proofs are plain-RUP LRAT, `cake_lpr` can be dropped in as a third, formally-verified verification pass.

Repository https://github.com/tanyongkiam/cake_lpr

CakeML project <https://cakeml.org>

Read to understand it Y. K. Tan, M. Heule, M. O. Myreen, “`cake_lpr`: Verified Propagation Redundancy Checking in CakeML” (TACAS 2021), <https://cakeml.org/tacas21.pdf>

2 Methodology (not a package, but essential to understand)

2.1 Cube-and-Conquer

The parallelisation strategy behind `cubes.txt`, `deepen.py`, and the per-cube UNSAT proofs: split (“cube”) the search space into many sub-problems, then conquer each with a full CDCL solver. Understanding this explains why there are 98,536 independent proofs and why coverage (`check_coverage.py`, exact mass = 1) is a necessary second ingredient.

Paper M. Heule, O. Kullmann, S. Wieringa, A. Biere, “Cube and Conquer: Guiding CDCL SAT Solvers by Lookaheads” (HVC 2011).

Author hub (many related papers) Marijn Heule, <https://www.cs.cmu.edu/~mheule/>

2.2 Lean 4 + mathlib — theorem prover (formalization layer, in progress)

The proof assistant hosting the project’s formalization layer (`Conway.lean/` in this folder, Lean v4.30.0-rc2 + mathlib): theorem T1 (the 98,536 cubes cover the space, replacing `check_coverage.py` in the trusted base) and, later, theorem T2 (the CNF encoding is faithful: UNSAT $\Rightarrow$ no σ -invariant srg). `cake_lpr` keeps discharging the per-cube UNSAT hypotheses; Lean proves the reasoning around them.

Lean website <https://lean-lang.org/>

mathlib (community math library) <https://leanprover-community.github.io/>

Read to understand it “Theorem Proving in Lean 4”, https://lean-lang.org/theorem_proving_in_lean4/

3 Compression and infrastructure

3.1 Zstandard (zstd)

The compressor for the 98,536 *.lrat.zst proofs and the referee_kit.tar.zst supplement; verify_final.sh decompresses on the fly.

Website <https://facebook.github.io/zstd/>

Repository <https://github.com/facebook/zstd>

3.2 Slurm Workload Manager

The HPC batch scheduler that ran the cube array on the cluster (o7_conquer.slurm, SLURM_TMPDIR).

Documentation <https://slurm.schedmd.com/>

3.3 Apptainer / Singularity (and Docker)

Containers appear in *two separate* roles here. (1) **Past — proof generation (finished)**. Singularity was the container runtime on the HPC cluster (module load singularity) that ran the cube-and-conquer solve; that compute is done and is not repeated. (2) **Planned — reproducible verification**. A small Docker/Apptainer image bundling PySAT + CaDiCaL + the checkers, so a referee can re-run the *verification* (not the search) with one command (the container/ folder). Note: Apptainer is simply the renamed Singularity, and it can also run Docker images.

Apptainer (current name of Singularity) <https://apptainer.org/>

Docker (alternative) <https://www.docker.com/>

3.4 Python 3

Runs the encoder, the exact-arithmetic coverage checker, and the aggregation scripts; the refvenv virtual environment holds PySAT.

Website <https://www.python.org/>